

Secure Enterprise RAG

Building Trusted Vectorized Knowledge Systems for DevSecOps

Ananda Kumar Dey
Quadrant Technologies LLC

Fragmented Knowledge Risks

Disconnected systems slow retrieval and raise enterprise risk.



Fragmented sources

Docs, runbooks, security artifacts, and deployment data live in disconnected tools.



Low retrieval precision

Keyword search misses relevant context and fails to surface the right answer quickly.



Operational impact

Slow response degrades incident handling, security posture, and service reliability.

Fragmented knowledge is a security and reliability risk — not just an inconvenience.

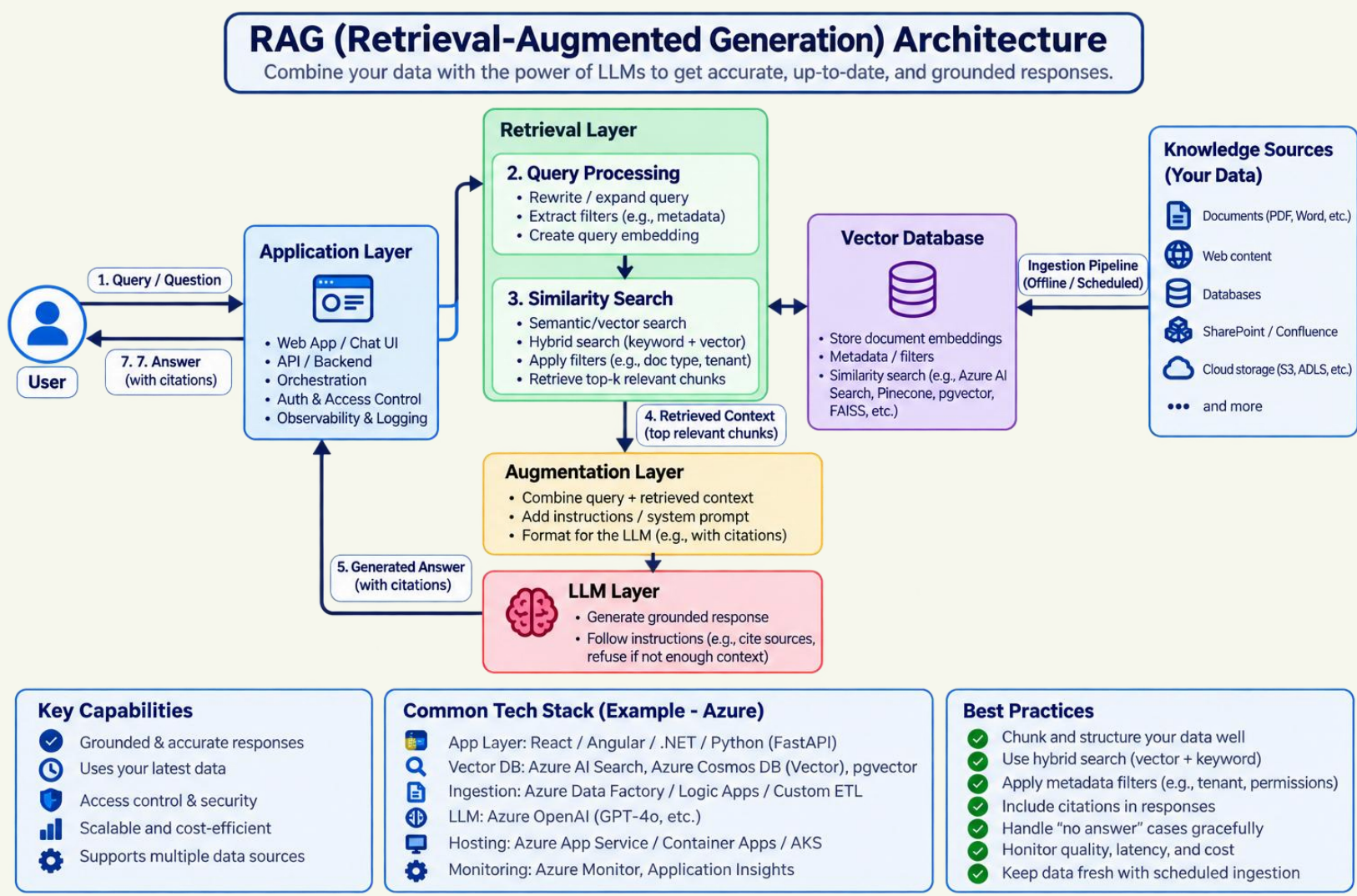
Session Agenda

- 1 RAG architecture fundamentals
- 2 Security and governance integration
- 3 Grounded retrieval and evaluation
- 4 Advanced patterns and production lessons



RAG Architecture Fundamentals

RAG Architecture Fundamentals



Ingestion Is Where Governance Starts

Production RAG begins with disciplined source intake, chunking, metadata, and pre-embedding controls.



Source connectors

Connect Confluence, GitHub, Jira, S3, ServiceNow, and internal wikis with governed, repeatable pipelines.



Chunking strategy

Tune semantic or fixed-size chunks and overlap to balance context fidelity with retrieval recall.



Metadata enrichment

Attach classification, lineage, owner, and sensitivity metadata to support filtering and auditability.



PII & secret detection

Scan before embedding so sensitive data never leaks into vector stores or downstream retrieval.

Production Retrieval Needs Strong Foundations

Model choice, vector store design, and index hygiene determine relevance, latency, and operational trust.



Embeddings

Use models like text-embedding-3-large and BGE-M3 to improve relevance for technical corpora.

- **Better semantic recall**
- **Model quality matters**



Vector stores

Assess Pinecone, Weaviate, pgvector, and OpenSearch for latency, filtering, and access control.

- **Policy-aware filtering**
- **Secure tenant controls**



Index hygiene

Run versioned indexes, deletion propagation, and stale-document TTLs as core production controls.

- **Propagate deletions fast**
- **Expire stale content**

Choosing a Vector Store

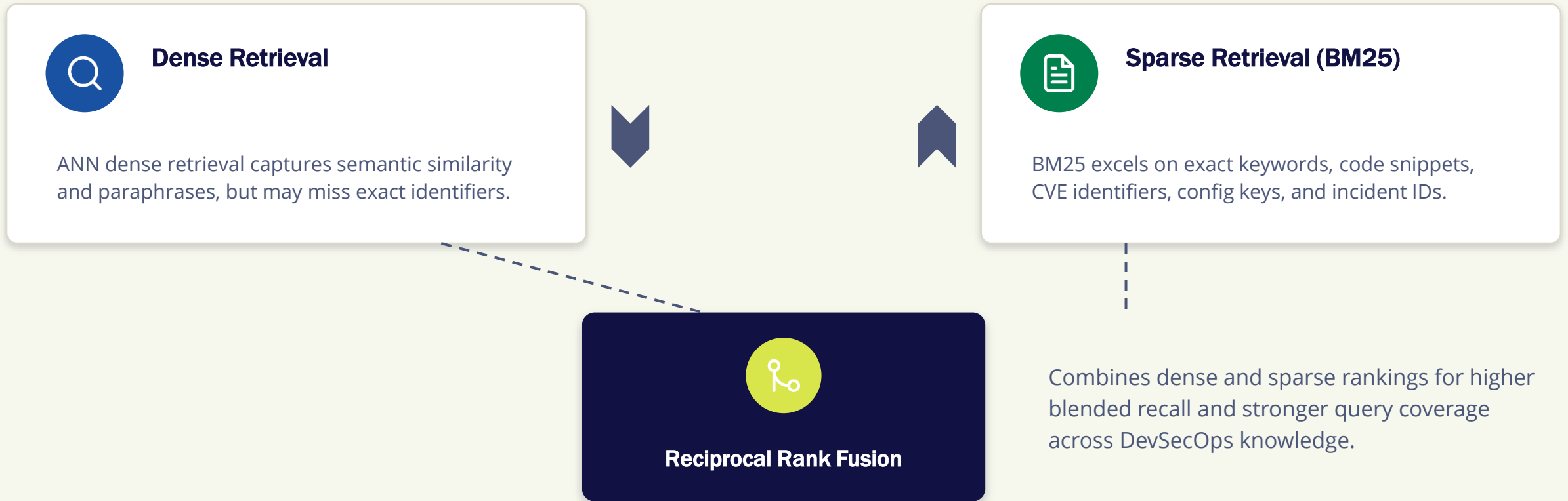
Trade-offs across the options named in the architecture — weigh hybrid search, filtering, and ops model together.

Vector Store	Hosting	Hybrid Search	Metadata Filtering	Best Fit
Pinecone	Fully managed	Native sparse + dense	Yes — metadata index	Fastest time-to-production
Weaviate	Managed or self-hosted	BM25 + vector (native)	Yes — GraphQL filters	Flexible deployment model
pgvector	Self-hosted (Postgres ext.)	Via tsvector + vector	Yes — plain SQL WHERE	Teams already on Postgres
Azure AI Search	Fully managed (Azure)	Native hybrid + semantic	Yes — OData filters	Azure-native, enterprise RBAC

All four appear in the reference architecture's vector database layer — the right choice depends on existing infra and filtering needs, not raw benchmark speed.

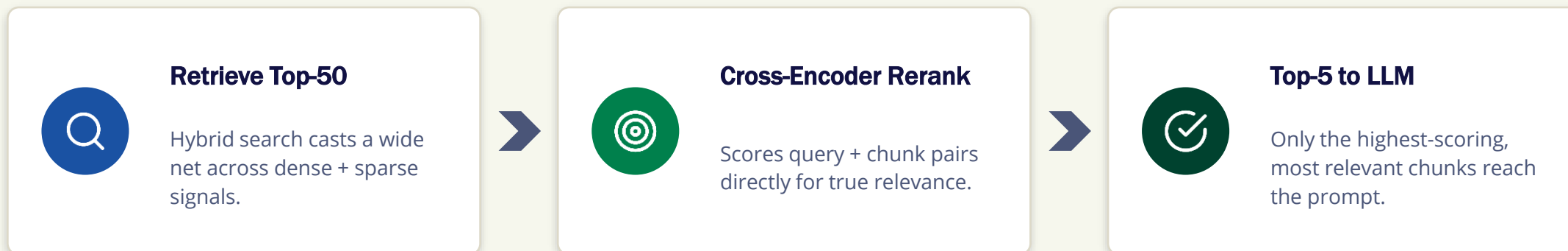
Hybrid Retrieval Outperforms Pure Vector Search

Dense and sparse signals work best together for DevSecOps queries that mix semantics with exact identifiers.

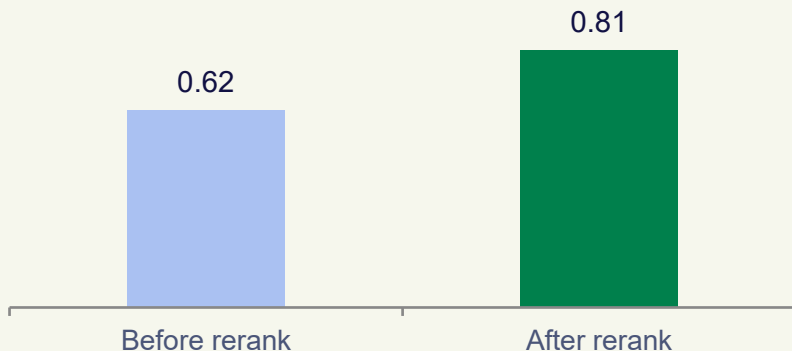


A Reranker Sharpens the Final Ranking

Retrieve broad, then rerank narrow — cross-encoders catch relevance signals embeddings miss.



RECALL@5



Worth knowing

- Common models: bge-reranker-large, Cohere Rerank, cross-encoder/ms-marco-MiniLM.
- Adds roughly 50–150ms — run it only on the top-k candidates, never the full index.
- Biggest win on ambiguous or multi-topic queries where embeddings alone rank loosely related chunks too high.

| Security by Design

Authorization at Retrieval Time

Security follows the full path from identity to filtered retrieval and authorized response.



Authorization must be enforced at retrieval, not just the UI layer — pre-filtering on metadata attributes is the recommended pattern.

Protecting and Auditing the Corpus

Core controls secure content before, during, and after vectorization — and leave a reviewable trail.

Protect the corpus



Encryption

AES-256 at rest, TLS 1.3 in transit, KMS-managed keys.



Secret Scanning

Gitleaks / Trufflehog catch exposed credentials pre-ingestion.



Classification

Tag content Public, Internal, Confidential, or Restricted.



Retention & Deletion

Tombstoning and GDPR-aligned deletion across vector stores.

Audit every retrieval



Query Logs

User identity, timestamp, query text, and top-k chunk IDs.



Response Logs

Generated text, model version, temperature, cited sources.



Immutable Trails

S3 Object Lock and CloudTrail for tamper-resistant records.



Anomaly Alerting

Abnormal retrieval patterns routed into SIEM workflows.



Grounded Retrieval Builds Trust

Controls That Reduce Hallucinations

Trusted sources, citation enforcement, and human review keep answers tied to verifiable evidence.



Trusted Sources

Ingest only reviewed, authoritative content. Exclude raw chat logs and unverified material.



Citation Enforcement

Require answers to cite retrieved chunks. Reject responses without grounded evidence.



Human Oversight

Route low-confidence or high-risk responses to review before they reach users.

Measure Retrieval, Not Guesswork

Use grounded metrics to isolate retrieval issues before tuning generation quality.



Top-K Quality

Precision and recall measure retrieval quality directly, independent of generation.



MRR Ranking

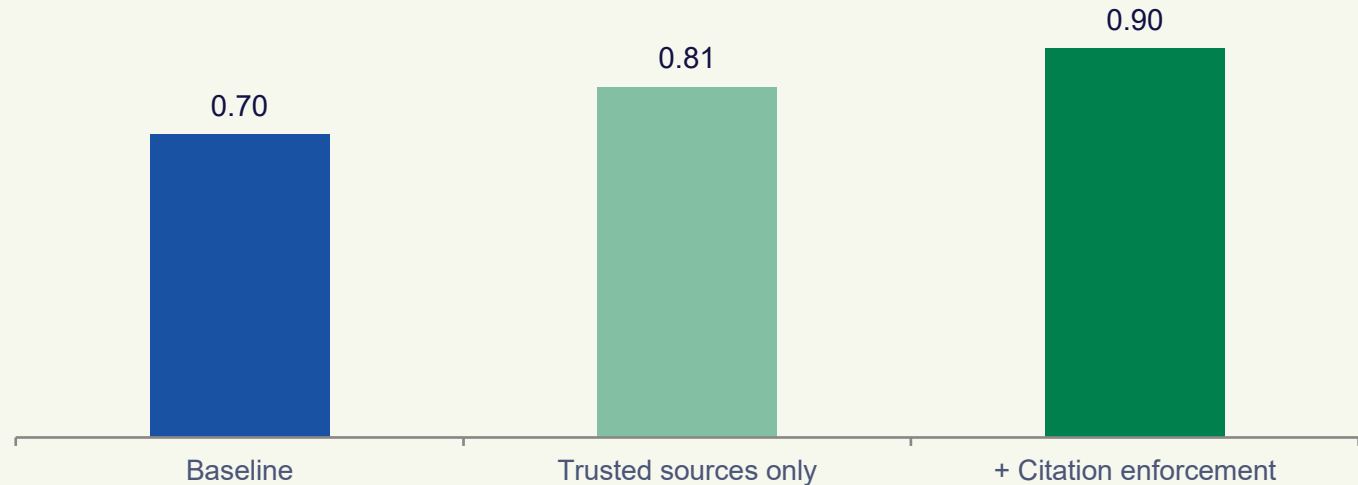
Shows whether the best chunk appears early or stays buried in the ranked list.



RAGAS Core

Tracks faithfulness, answer relevancy, and context recall on representative queries.

RAGAS Faithfulness — Before vs. After Citation Enforcement



Industry reference ranges (validate against your own golden dataset): hybrid retrieval can lift recall@10 by ~10–20 points; end-to-end latency budget runs ~1.1–3.3s (embedding + search + rerank + generation).

| Advanced RAG Patterns

Beyond Flat Similarity Search

Graph traversal, agents, and live tools extend enterprise RAG as query complexity grows.

Increasing sophistication →



Graph Retrieval

Maps entities and links to trace dependencies, owners, incidents, and CVEs across records.



Agentic RAG

Agents decompose hard questions, plan multi-step retrieval, and synthesize diverse evidence.



Tool-Augmented RAG

Agents call APIs and pull live system state alongside indexed enterprise knowledge.

Production Lessons

Enterprise deployments reward disciplined data, controls, engineering, and measurement.



Fix data before tuning models

Retrieval quality depends first on source accuracy, completeness, and structure.



Enforce access in retrieval

UI-only controls aren't enough — authorization must govern what the retriever can fetch.



Measure before optimizing

Set baselines first, then tune chunking, overlap, prompts, and models with evidence.



Run RAG like software

Apply CI/CD, versioning, observability, and incident response to the whole pipeline.

Enterprise RAG Recap

Trusted systems combine security, hybrid retrieval, continuous evaluation, and grounded oversight.



Secure pipeline



Hybrid wins



Evaluate always



Design for trust

NEXT STEPS



Audit and classify

Review sources for boundaries, sensitivity, ownership, and data quality risk.



Build a golden set

50–100 representative queries with expert-validated expected retrievals.



Launch hybrid retrieval

Hybrid search with retrieval-time authorization as baseline, then measure.



Thank You

Ananda Kumar Dey